From Wallet Seed to Ethereum Address: Hashing, Key Derivation, and Cryptographic Primitives

Homework 2 – Cryptography and Blockchain Architectures

Department of Computer Science and Engineering

October 2026

Abstract

Modern digital wallets (such as MetaMask, Rabby, Exodus, and Rainbow) implement a multi-stage cryptographic architecture designed to reconcile human usability with strict mathematical security. Rather than relying on a single monolithic algorithm, these systems assemble an ordered pipeline of distinct cryptographic primitives, key derivation standards, and encoding schemes. This paper examines the complete deterministic pipeline from a BIP-39 mnemonic sentence to an EIP-55 checksummed Ethereum address. We establish rigorous theoretical distinctions between key generation, key derivation, asymmetric cryptography, cryptographic hashing, checksums, and encoding. Furthermore, we analyze the statistical properties of each stage, contextualize the historical evolution of the underlying standards from 1977 to 2016, and clarify the structural divergence between the Merkle-Damgård construction of SHA-2 and the sponge construction of Keccak-256 and NIST SHA-3.

1. Fundamental Conceptual Distinctions

Before examining the specific Ethereum key management pipeline, it is necessary to establish rigorous definitions for the five foundational operations involved in wallet architecture. Confusing these functional categories is a frequent source of error in security analysis.

- **Key Generation vs. Key Derivation:** *Key generation* is a stochastic process that extracts raw randomness (entropy) from a physical or operating-system source (a True Random Number Generator or Cryptographically Secure Pseudo-Random Number Generator) to create an independent cryptographic secret. *Key derivation*, conversely, is a strictly deterministic mathematical process. Given an initial master secret and a set of structural parameters, a Key Derivation Function (KDF) computes secondary keys. If the initial secret is preserved, the entire tree of derived keys can be reproduced identically without storing individual leaf keys.

- **Public-Key (Asymmetric) Cryptography:** A mathematical framework relying on trapdoor one-way functions (such as the Elliptic Curve Discrete Logarithm Problem) to establish a pair of mathematically linked keys: a private key k kept secret, and a public key K shared openly. In distributed ledgers, it serves two roles: digital signatures (proving ownership and authorizing state transitions without revealing k) and identity representation.

- **Cryptographic Hashing:** A deterministic function $H : \{0,1\}^* \to \{0,1\}^n$ that maps an arbitrary-length input message to a fixed-size bit string (the digest). A secure cryptographic hash function must satisfy three core properties: *preimage resistance* (infeasibility of finding x given $H(x)$), *second-preimage resistance* (infeasibility of finding $x' \neq x$ such that $H(x) = H(x')$), and *collision resistance* (infeasibility of finding any pair (x, x') mapping to the same digest).

- **Checksums:** Redundancy mechanisms computed from a data payload for the explicit purpose of detecting accidental errors introduced during transmission, transcription, or storage. Unlike cryptographic hashes, which defend against adversarial tampering, checksums defend against benign human or channel errors (such as mistyping a character in an address).

- **Encoding:** A reversible, non-cryptographic bijection that translates data from one representation format to another (e.g., raw binary bytes to hexadecimal characters, Base58, or natural language words) to improve compatibility or human readability. Encoding involves no secret key and provides zero confidentiality.

2. The Ethereum Wallet Pipeline and Statistical Properties

The transformation from a user-facing backup phrase to an on-chain Ethereum address proceeds through six sequential stages, illustrated in Figure 1.

2.1 BIP-39: Mnemonic Sentence and Seed Generation

Raw cryptographic keys (256-bit integers) are prone to human transcription errors. BIP-39 standardizes the encoding of raw entropy into a sequence of natural language words and its subsequent expansion into a binary master seed [6].

1. **Entropy and Checksum:** For a standard 12-word phrase, the wallet generates 128 bits of initial entropy

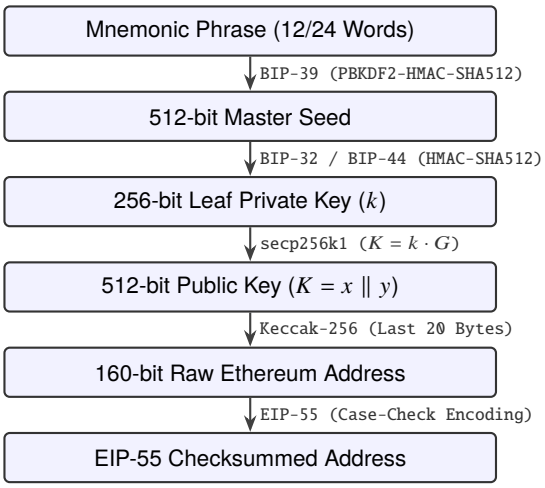


Figure 1: Architectural pipeline of an Ethereum HD wallet from initial mnemonic entropy to the final EIP-55 address.

(ENT). For a 24-word phrase, 256 bits are generated. A checksum is computed by taking the first $ENT/32$ bits of the SHA-256 digest of the entropy (4 bits for 128-bit entropy; 8 bits for 256-bit entropy) and appending them to the initial bit string.

2. **Word Mapping:** The concatenated string (132 bits for 12 words) is partitioned into 11-bit segments. Since $2^{11} = 2048$, each 11-bit integer indexes a standardized dictionary of 2,048 words.

3. **Seed Derivation:** The mnemonic string is passed into the Password-Based Key Derivation Function 2 (**PBKDF2**) using **HMAC-SHA512** as the underlying pseudo-random function. The mnemonic phrase acts as the password, while the string `"mnemonic"` concatenated with an optional user passphrase serves as the salt. PBKDF2 executes 2,048 iterations to produce a 512-bit (64-byte) master seed.

Statistical Properties: The initial entropy must follow a discrete uniform distribution over $\{0,1\}^{128}$ (or $\{0,1\}^{256}$). Any statistical bias in the random number generator reduces the effective keyspace. Although PBKDF2 stretches the input into a 512-bit seed that is computationally indistinguishable from uniform random noise, the actual *information-theoretic entropy* (Shannon entropy) of a 12-word seed remains bounded at 128 bits (excluding the optional passphrase).

2.2 BIP-32: Hierarchical Deterministic Key Derivation

Early cryptocurrency wallets generated unrelated random keys ("Just-a-Bunch-of-Keys"), requiring frequent backups. BIP-32 transforms the 512-bit seed into a deterministic tree of key pairs [5].

The 512-bit seed is hashed via HMAC-SHA512: the left 256 bits form the Master Private Key (k_m), and the right 256 bits form the Master Chain Code (c_m). A parent extended key $(k_{\text{par}}, c_{\text{par}})$ derives up to 2^{32} child keys via a Child Key Derivation (CKD) function indexed by i:

- **Normal Derivation ($i < 2^{31}$):** Computes HMAC-SHA512$(c_{\text{par}}, K_{\text{par}} \parallel i)$ using the parent *public* key. This allows watch-only servers holding the extended public key (`xpub`) to generate child public addresses without access to private keys. However, leaking a single child private key alongside the `xpub` allows an adversary to algebraically recover the parent private key.

- **Hardened Derivation ($i \geq 2^{31}$, denoted i'):** Computes HMAC-SHA512$(c_{\text{par}}, \texttt{0x00} \parallel k_{\text{par}} \parallel i)$ using the parent *private* key. This severs the algebraic relationship between parent public keys and child private keys, isolating branches against compromise.

Statistical Properties: Because HMAC-SHA512 operates as a Pseudo-Random Function (PRF), every derived child key and chain code is statistically independent and uniformly distributed across the 256-bit domain. Knowledge of one child key yields zero statistical correlation to sibling keys provided the chain code remains secret.

2.3 BIP-44: Standard Multi-Account Derivation Paths

While BIP-32 defines the branching algebra, BIP-44 establishes a universal five-level semantic schema so that different wallet implementations (e.g., migrating from MetaMask to Rabby) derive identical accounts [7]:

$$m/\text{purpose}'/\text{coin_type}'/\text{account}'/\text{change}/\text{address_index} \tag{1}$$

For Ethereum, the standard derivation path is `m/44'/60'/0'/0/x`, where `44'` denotes BIP-44 adherence, `60'` is Ethereum's SLIP-0044 coin identifier, `0'` is the hardened account index, `0` designates the external chain, and `x` is the sequential address index.

2.4 secp256k1: Elliptic-Curve Public-Key Cryptography

The leaf node of the BIP-44 path yields a 256-bit private key $k \in [1, n-1]$, where $n \approx 1.158 \times 10^{77}$ is the prime order of the elliptic curve cyclic group. Ethereum employs the **secp256k1** Koblitz curve [3], defined over the finite prime field $\mathbb{F}_p$ (with $p = 2^{256} - 2^{32} - 977$) by the short Weierstrass equation:

$$y^2 \equiv x^3 + 7 \pmod{p} \tag{2}$$

The corresponding public key K is derived via elliptic curve scalar multiplication:

$$K = k \cdot G \tag{3}$$

where G is the standardized generator point. The public key $K = (x, y)$ consists of two 256-bit coordinates serialized as a 512-bit (64-byte) uncompressed byte array.

Statistical Properties: Security rests on the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**. While forward computation $K = k \cdot G$ requires $O(\log k)$ group operations via the double-and-add algorithm, inverting the mapping requires $O(\sqrt{n}) \approx 2^{128}$ operations using Pollard's rho algorithm. The generated points (x, y) are uniformly distributed across the subgroup generated by G, exhibiting no structural clustering.

2.5 Keccak-256: Ethereum Address Derivation

To minimize state storage overhead, Ethereum compresses the 64-byte public key by hashing the raw coordinate concatenation $(x \parallel y)$ via **Keccak-256** (omitting the `0x04` uncompressed prefix byte) [8]:

$$\text{Digest} = \text{Keccak-256}(x \parallel y) \in \{0,1\}^{256} \tag{4}$$

The Ethereum address consists of the **last 20 bytes** (160 bits) of this digest, prefixed with `0x` to form a 40-character hexadecimal string.

Statistical Properties: Keccak-256 satisfies the **strict avalanche criterion**: flipping a single bit in $(x \parallel y)$ alters each output bit independently with probability $p = 0.5$. Truncating the digest to 160 bits bounds the address space to $2^{160} \approx 1.46 \times 10^{48}$. By the Birthday Paradox, finding an arbitrary collision requires $\approx 2^{80}$ operations, while a targeted second-preimage attack against a specific address requires 2^{160} operations.

2.6 EIP-55: Mixed-Case Checksum Encoding

Ethereum initially launched without address checksums. To introduce error detection without altering the 20-byte raw address length, Vitalik Buterin proposed **EIP-55** in 2016 [10]. The algorithm hashes the 40-character lowercase ASCII address via Keccak-256: $H = \text{Keccak-256}(\text{address}_{\text{lower}})$. For each character position $i \in [0, 39]$, if the character is alphabetical (`a`–`f`), it is capitalized if and only if the i-th hexadecimal nibble of H has a value ≥ 8 (i.e., its most significant bit is 1).

Statistical Properties: Assuming a uniform distribution of hexadecimal characters, each position has a $\frac{6}{16} = 0.375$ probability of being a letter (`a`–`f`) and a $\frac{10}{16} = 0.625$ probability of being a numeral (`0`–`9`). Across 40 characters, the expected number of letters is:

$$E[\text{letters}] = 40 \times \frac{6}{16} = 15 \text{ characters} \tag{5}$$

Since each alphabetical character encodes exactly 1 bit of binary casing state, EIP-55 embeds an average of 15 bits of checksum entropy. The probability that a randomly corrupted address accidentally satisfies the EIP-55 casing rules (false negative rate) is:

$$P(\text{undetected error}) \approx 2^{-15} = \frac{1}{32,768} \approx 0.00305\% \tag{6}$$

3. Temporal and Historical Context

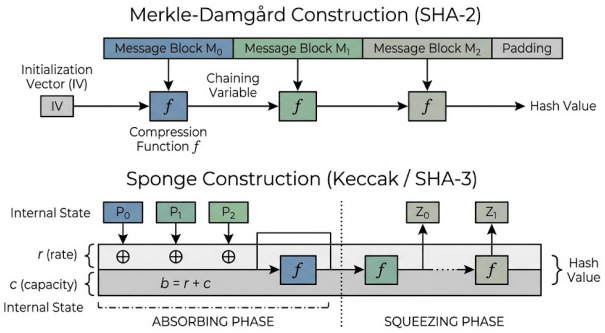
The Ethereum wallet pipeline represents the convergence of nearly four decades of cryptographic standardization, summarized chronologically in Table 1.

3.1 Architectural Distinction: SHA-2 vs. Keccak / SHA-3

Although both belong to the cryptographic hash family, **SHA-2** and **Keccak/SHA-3** rely on fundamentally distinct internal architectures, and Ethereum's **Keccak-256** is not identical to NIST's **SHA3-256**.

Table 1: Chronological Evolution of Core Wallet Technologies

Year	Technology	Historical Role and Milestone
1977	RSA	Rivest, Shamir, and Adleman introduce practical public-key cryptography [1]. Later superseded in blockchains by compact ECC (1985).
2001	SHA-2 / SHA-512	NIST publishes FIPS 180-2 [2]. Forms the HMAC backbone for BIP-32 and BIP-39.
2012	BIP-32	Pieter Wuille introduces hierarchical deterministic (HD) key trees [5].
2013	BIP-39	Palatinus et al. standardize human-readable mnemonic phrases and PBKDF2 seed stretching [6].
2014	BIP-44	Palatinus and Rusnak define multi-coin, multi-account HD path conventions [7].
2015	SHA-3 (Keccak)	NIST standardizes Keccak as FIPS 202 following a public competition [9].
2016	EIP-55	Buterin and Beregszaszi introduce mixed-case hexadecimal address checksums [10].



1. **Merkle-Damgård vs. Sponge Construction:** SHA-2 (SHA-256, SHA-512) uses the *Merkle-Damgård construction* with a Davies-Meyer compression function, updating an internal state via modular additions and bitwise rotations. This structure is inherently susceptible to *length-extension attacks*, requiring HMAC wrappers in BIP-32 and BIP-39. Conversely, Keccak/SHA-3 utilizes a *Sponge construction* [4] operating on a 1600-bit state ($b = r + c$, partitioned into bit rate r and capacity c). Input blocks are "absorbed" via XOR and processed through 24 rounds of an unkeyed permutation (Keccak-$f[1600]$), after which the output is "squeezed" out. The hidden capacity c makes sponge functions natively immune to length-extension attacks.

2. **Domain-Separation Padding Divergence:** Ethereum launched its Frontier network in July 2015, one month prior to NIST's final publication of FIPS 202 in August 2015 [9]. In the final standard, NIST appended a 2-bit domain separation suffix (`01`) prior to the multi-rate padding rule (pad10*1). Consequently, original **Keccak-256** (used by Ethereum) pads the message with byte `0x01`, whereas standardized **NIST SHA3-256** pads

with byte `0x06`. Inputting the same public key into both functions produces entirely different digests.

4. Synthesis: Necessity of a Multi-Algorithm Pipeline

A critical engineering question is why Ethereum combines six separate standards instead of employing a single cryptographic primitive across the entire wallet pipeline. In cryptographic systems design, distinct mathematical structures are required to satisfy orthogonal security and usability constraints (Table 2).

Table 2: Functional Separation Across the Wallet Pipeline

Component	**Domain**	**Architectural Justification**
BIP-39 / 32 / 44	Key Management	Fast hashes allow brute-forcing of weak mnemonics; PBKDF2 enforces key stretching. HMAC-SHA512 combined with elliptic algebra enables hierarchical trees and public-only derivation (`xpub`).
secp256k1	Asymmetric Crypto	Symmetric hashes cannot sign messages without exposing the secret. Elliptic curve trapdoors ($K = k \cdot G$) enable non-repudiable ECDSA transaction signatures.
Keccak-256	Hashing	Compresses 64-byte algebraic curve points into unstructured 20-byte identifiers and conceals K until the first outgoing transaction.
EIP-55	Error Detection	Raw hex strings lack redundancy; any typo burns funds. Case modulation embeds a 15-bit checksum with zero byte overhead.

By decoupling human mnemonic encoding (BIP-39), tree derivation (BIP-32/44), asymmetric trapdoor operations (secp256k1), one-way compression (Keccak-256), and transcription error detection (EIP-55), the Ethereum wallet architecture achieves defense-in-depth and modular interoperability [11].

References

[1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.

[2] National Institute of Standards and Technology (NIST), "FIPS PUB 180-2: Secure Hash Standard (SHS)," U.S. Department of Commerce, Aug. 2001.

[3] Certicom Research, "SEC 2: Recommended Elliptic Curve Domain Parameters (Version 2.0)," *Standards for Efficient Cryptography*, Jan. 2010.

[4] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, "The Keccak Reference (Version 3.0)," Submission to the NIST SHA-3 Competition, Jan. 2011.

[5] P. Wuille, "BIP-32: Hierarchical Deterministic Wallets," *Bitcoin Improvement Proposals*, Feb. 2012.

[6] M. Palatinus, P. Rusnak, A. Voisine, and S. Bowe, "BIP-39: Mnemonic code for generating deterministic keys," *Bitcoin Improvement Proposals*, Sep. 2013.

[7] M. Palatinus and P. Rusnak, "BIP-44: Multi-Account Hierarchy for Deterministic Wallets," *Bitcoin Improvement Proposals*, Apr. 2014.

[8] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, 2014.

[9] National Institute of Standards and Technology (NIST), "FIPS PUB 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," U.S. Department of Commerce, Aug. 2015.

[10] V. Buterin and A. Beregszaszi, "EIP-55: Mixed-case checksum address encoding," *Ethereum Improvement Proposals*, Jan. 2016.

[11] A. M. Antonopoulos and G. Wood, *Mastering Ethereum: Building Smart Contracts and DApps*. O'Reilly Media, 2018.